CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)

Practical File – ODD Semester 2026–27

EnrollNo: D25DIT078

Name:Jay Kansara

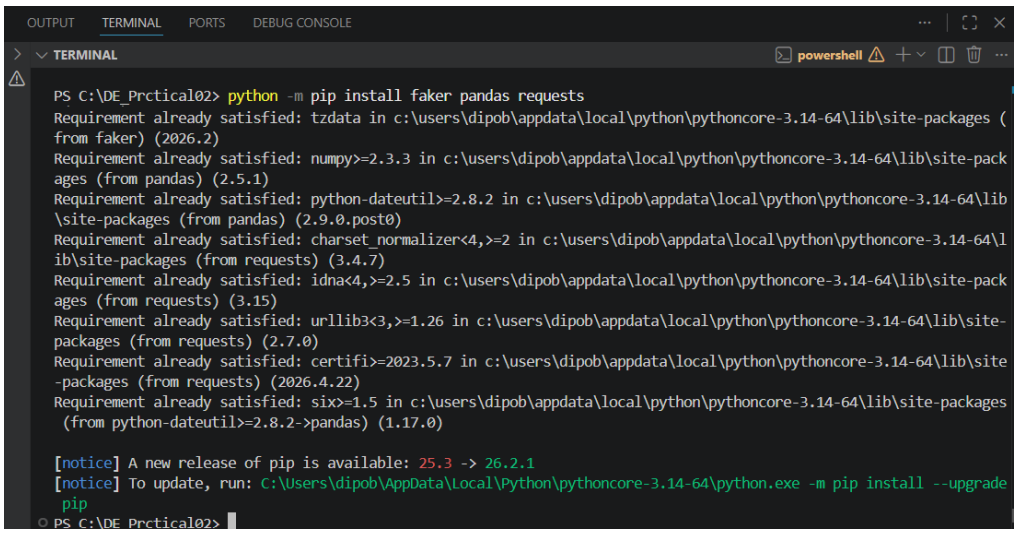
Practical 02

Problem Definition: Programmatically interface with diverse file-based, API-driven, and relational source database environments. Students will write generation engines to simulate transactional workloads, profile the incoming files to automatically discover schemas, isolate data quality violations (such as null entry leaks and format divergence), and establish appropriate operational ingestion criteria.

Dataset/Test Data: Programmatically simulated customer profile tables, semi-structured API transaction logs containing nested objects, and unstructured configuration text files.

Tools/Technology to Be Used: Python 3.x, Faker library, Pandas, Requests HTTP library, SQLite/PostgreSQL, and Git.

Step 1: Install Required Libraries



Step 2: Generate Customer Dataset

data_ingestion.py

```python
from faker import Faker
import pandas as pd

fake = Faker()

customers = []

for i in range(20):
    customers.append({
        "CustomerID": i + 1,
        "Name": fake.name(),
        "City": fake.city(),
        "Email": fake.email()
    })

df = pd.DataFrame(customers)

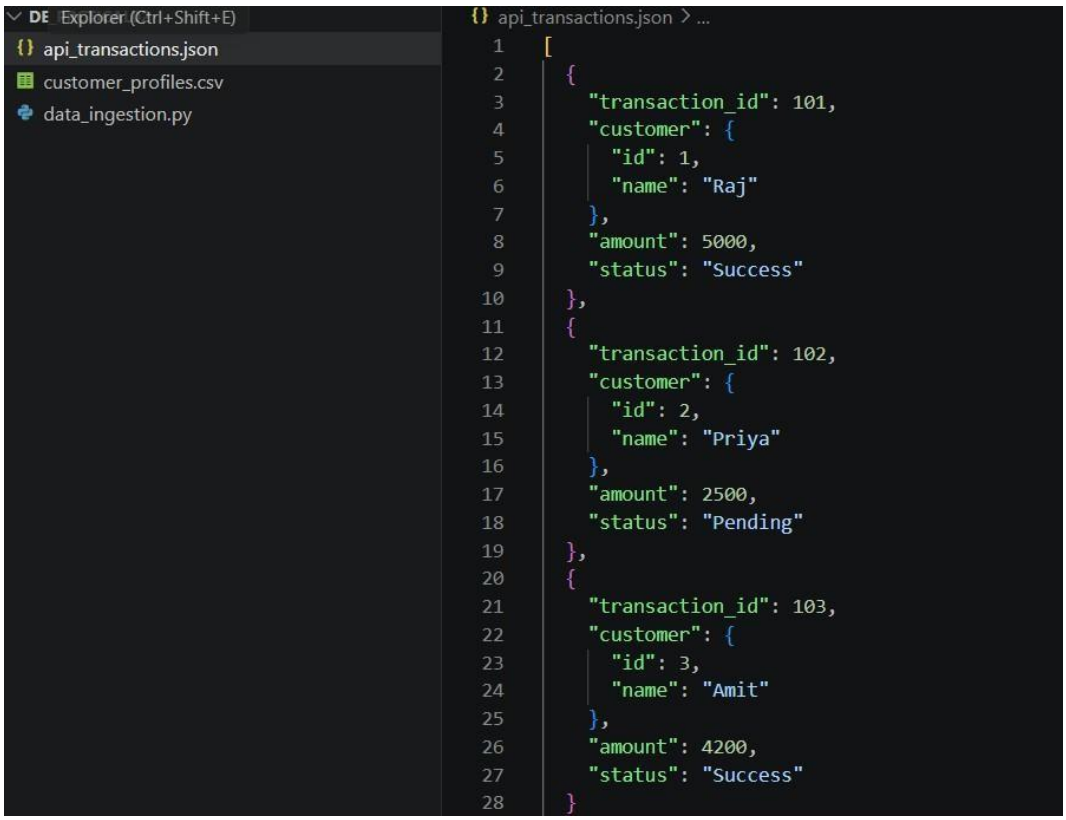
df.to_csv("customer_profiles.csv", index=False)

print(df.head())
```

```
PS C:\DE_Prctical02> python data_ingestion.py
   CustomerID               Name           City                        Email
0           1    Kristina Raymond      Harveyton  aguilarkathryn@example.com
1           2          Amy Holder   North Thomas    carolynlopez@example.com
2           3       Jennifer Gray  West Nicholas     riveradavid@example.org
3           4  Christopher Guerra      Alexmouth   jonathanmoore@example.org
4           5      Justin Buckley      Rojasside     nicolewhite@example.org
PS C:\DE_Prctical02>
```

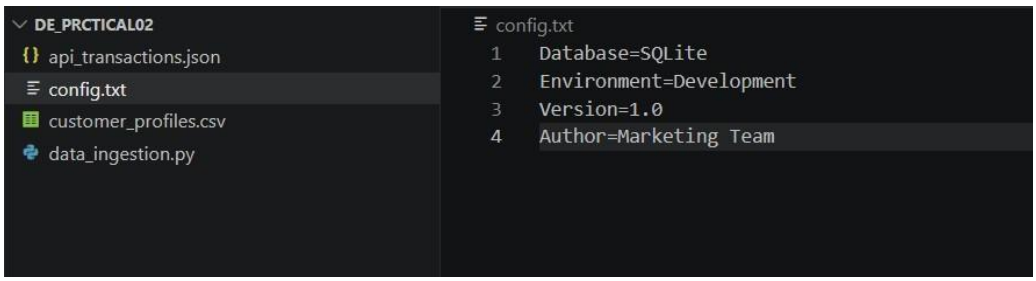
Step 3: Create the API Transaction JSON File

This JSON file represents API transaction data. It contains nested customer information along with transaction details such as transaction ID, amount, and status.



Step 4: Create the Configuration File

This file stores basic application configuration settings, such as the database type and environment.



Step 5: Read All Three Data Sources

data_ingestion.py code:

```
import pandas as pd
import json
csv_data = pd.read_csv("customer_profiles.csv")
print("========== CUSTOMER DATA ==========")
print(csv_data.head())
with open("api_transactions.json", "r") as file:
    api_data = json.load(file)
print("\n========== API TRANSACTIONS ==========")
print(api_data)
with open("config.txt", "r") as file:
    config = file.read()
print("\n========== CONFIGURATION ==========")
print(config)
```

```
PS C:\DE_Prctical02> python data_ingestion.py
========== CUSTOMER DATA ==========
   CustomerID               Name         City                       Email
0           1       Gregory Cole  East Austin         ryan17@example.org
1           2         Julia Hill   Paynemouth     jamessutton@example.net
2           3  Stephanie Castillo     West Amy         kelly74@example.org
3           4         Haley Page    Bryanbury         tracy87@example.org
4           5     Tiffany Mueller  South Kerry  sparksbenjamin@example.net

========== API TRANSACTIONS ==========
[{'transaction_id': 101, 'customer': {'id': 1, 'name': 'Raj'}, 'amount': 5000, 'status': 'Success'
}, {'transaction_id': 102, 'customer': {'id': 2, 'name': 'Priya'}, 'amount': 2500, 'status': 'Pend
ing'}, {'transaction_id': 103, 'customer': {'id': 3, 'name': 'Amit'}, 'amount': 4200, 'status': 'S
uccess'}]

========== CONFIGURATION ==========
Database=SQLite
Environment=Development
Version=1.0
Author=Marketing Team
PS C:\DE_Prctical02>
```

Step 6: Data Profiling

print("\n========== DATASET INFORMATION ==========")

csv_data.info()

print("\n========== STATISTICAL SUMMARY ==========")

print(csv_data.describe(include="all"))

```
PS C:\DE_Prctical02> python data_ingestion.py

========== DATASET INFORMATION ==========
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   CustomerID  20 non-null     int64
 1   Name        20 non-null     str
 2   City        20 non-null     str
 3   Email       20 non-null     str
dtypes: int64(1), str(3)
memory usage: 772.0 bytes

========== STATISTICAL SUMMARY ==========
        CustomerID          Name         City              Email
count     20.00000            20           20                 20
unique         NaN            20           20                 20
top            NaN  Gregory Cole  East Austin  ryan17@example.org
freq           NaN             1            1                  1
mean      10.50000           NaN          NaN                NaN
std        5.91608           NaN          NaN                NaN
min        1.00000           NaN          NaN                NaN
25%        5.75000           NaN          NaN                NaN
50%       10.50000           NaN          NaN                NaN
75%       15.25000           NaN          NaN                NaN
max       20.00000           NaN          NaN                NaN
```

Step 7: Check Missing Values

print("\n========== MISSING VALUES ==========")

print(csv_data.isnull().sum())

```
========== MISSING VALUES ==========
CustomerID    0
Name          0
City          0
Email         0
dtype: int64
```

Step 8: Check Duplicate Records

print("\n========== DUPLICATE RECORDS ==========")

print(csv_data.duplicated().sum())

```
========== DUPLICATE RECORDS ==========
0
PS C:\DE_Prctical02>
```

Step 9: Store Data in SQLite Database

```
========== STORING DATA INTO SQLITE ==========
Data stored successfully in SQLite database.
PS C:\DE_Prctical02>
```

Step 10: Read Data from SQLite

```
PS C:\DE_Prctical02> python read_database.py
========== DATA FROM SQLITE DATABASE ==========
    CustomerID              Name            City                    Email
0            1     Gregory Cole      East Austin         ryan17@example.org
1            2       Julia Hill       Paynemouth     jamessutton@example.net
2            3  Stephanie Castillo        West Amy        kelly74@example.org
3            4       Haley Page        Bryanbury        tracy87@example.org
4            5   Tiffany Mueller      South Kerry  sparksbenjamin@example.net
5            6  Patricia Fischer       Gordonberg     brownmanuel@example.net
6            7   Rachel Robinson  West Ericborough  brandiwilliams@example.com
7            8     Joseph Butler       Wilcoxport       pcalderon@example.org
8            9   Lacey Rodriguez     West Susanton       william35@example.net
9           10    James Martinez       Baileyfort       qwilliams@example.net
10          11   Kristen Mooney  West Zacharyview      ramosangela@example.net
11          12   Lawrence Flores         Lunaland        ronald47@example.com
12          13     Lisa Gonzalez    South Kristina      dixonsteven@example.com
13          14    Heather Willis      Williamsport     hinesshannon@example.com
14          15    Alexandra Long    East Coreyberg          qcombs@example.org
15          16     David Carroll       New Anthony          john31@example.com
16          17         Jason Lee       Port Martin       terrybrock@example.org
17          18     Jaclyn Warren          New Troy         james91@example.net
18          19       Jodi Dorsey       Leslieland  richardcallahan@example.com
19          20  Andrew Hernandez       Heatherton      nicolejones@example.net
PS C:\DE_Prctical02>
```

Step 11: Generate Data Quality Log

```
Quality log generated successfully.
PS C:\DE_Prctical02>
```

SUPPLEMENTARY PROBLEM

```
PS C:\DE_Prctical02> python validate_files.py
customer_profiles.csv is valid
PS C:\DE_Prctical02>
```

summary.md

```
summary.md > # Data Quality Summary > ## Result
# Data Quality Summary

## Objective
To validate and profile customer data before storing it into the database.

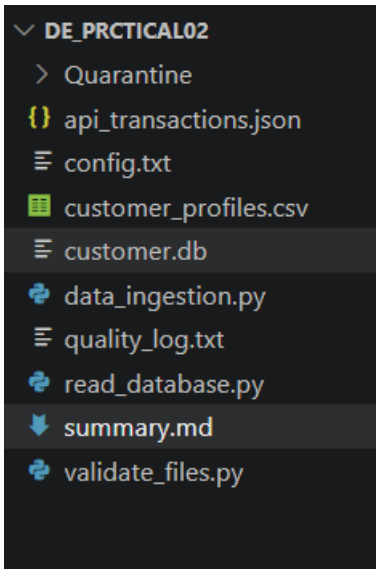
## Validation Performed

- Generated synthetic customer dataset using Faker
- Read CSV, JSON and configuration files
- Profiled dataset
- Checked missing values
- Checked duplicate records
- Stored validated data into SQLite
- Generated data quality log
- Validated incoming files

## Result

The dataset passed all validation checks successfully and was stored in the SQLite database.
```

Folder Structure



Key Questions

Q1. Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs.

Answer:
Schema drift can be managed using schema versioning, flexible parsers, metadata validation, optional fields, and automated schema detection before data ingestion.

Q2. What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records?

Answer:
Nested JSON objects provide flexibility, support changing schemas, and naturally represent hierarchical data. However, they may require more storage and are generally less efficient for complex relational queries than normalized SQL databases.

Q3. How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks?

Answer:
Use lightweight validation checks such as verifying required fields, detecting missing values, checking data types, and identifying duplicate records. Perform these validations in batches and log errors without stopping the entire ingestion process.